

Contenido

Introducción.....	2
Las aplicaciones Web	2
Modelo de arquitectura de servicios	5
Modelo Cliente/Servidor.....	6
Instalación y configuración de servidores Web	8
Modelo Relacional.....	14
Modelo de Datos.....	14
Diseño de Bases de Datos	19
Introducción a PHP	21
¿Qué es PHP?.....	21
Instalación y configuración de PHP sobre servidor Web	22
Fuentes.....	29

Introducción

Las aplicaciones Web

En un principio la Web era sencillamente una colección de páginas estáticas, documentos, etc., para su consulta o descarga. El paso inmediatamente posterior en su evolución fue la inclusión de un método para elaborar páginas dinámicas que permitieran que lo mostrado tuviese carácter dinámico (es decir, generado a partir de los datos de la petición). Este método fue conocido como CGI ("Common Gateway Interface") y definía un mecanismo mediante el que se podía pasar información entre el servidor y ciertos programas externos.

Los CGIs siguen utilizándose ampliamente; la mayoría de los servidores web permiten su uso debido a su sencillez. Además, dan total libertad para elegir el lenguaje de programación que se desea emplear.

El funcionamiento de los CGIs tenía un punto débil: cada vez que se recibía una petición, el servidor debía lanzar un proceso para ejecutar el programa CGI. Como la mayoría de CGIs estaban escritos en lenguajes interpretados, como Perl o Python, o en lenguajes que requerían "run-time environment", como Java o VisualBasic, el servidor se veía sometido a una gran carga. La concurrencia de múltiples accesos al CGI podía comportar problemas graves.

Por eso se empiezan a desarrollar alternativas a los CGIs que solucionaran el problema del rendimiento. Las soluciones llegan básicamente por 2 vías: 1) se diseñan sistemas de ejecución de módulos mejor integrados con el servidor, que evitan la instanciación y ejecución de varios programas, y 2) se dota a los servidores un intérprete de algún lenguaje de programación que permita incluir el código en las páginas de forma que lo ejecute el servidor, reduciendo el intervalo de respuesta.

Entonces se experimenta un aumento del número de arquitecturas y lenguajes que permiten desarrollar aplicaciones web. Todas siguen alguna de estas vías. Las más útiles y las más utilizadas son las que permiten mezclar los 2 sistemas: un lenguaje integrado que permita al servidor interpretar comandos "incrustados" en las páginas HTML y, además, un sistema de ejecución de programas mejor enlazado con el servidor, que no implique los problemas de rendimiento propios de los CGIs.

Una de las más potentes es la seguida por Sun Microsystems con su Java, integrado por 2 componentes; un lenguaje que permite la incrustación de código en las páginas HTML que el servidor convierte en programas ejecutables, JSP ("Java Server Pages" o "Páginas de Servidor de Java"), y un método de programación muy ligado al servidor, con un rendimiento superior a los CGIs, denominado "Java Servlet".

Otra tecnología de éxito y una de las más utilizadas es el lenguaje PHP. Se trata de un lenguaje interpretado que permite la incrustación de HTML en los programas,

con una sintaxis derivada de C y Perl. El hecho de ser sencillo y potente ha contribuido a hacer de PHP una herramienta muy apropiada para determinados desarrollos.

Existen otros métodos, a menudo vinculados a un Servidor web concreto, como mod_perl para Apache o RXML para Roxen.

Cada vez es más común que en cualquier tipo de entidad (empresa, asociación, negocio, autónomo, institución...) necesite una **solución informática ajustada a sus necesidades**. Y cada vez es más habitual que se nombre una “Aplicación Web” como la respuesta a estas necesidades

El abanico es tan amplio que es difícil concretar en un texto las infinitas posibilidades que existen. Aquí os dejamos algunos ejemplos

- Aplicaciones para la **gestión interna** y completa de la entidad (facturación, stock, clientes, usuarios, socios, contabilidad, fichar, gestión de personal...),
- **Herramientas de trabajo** diversas para entidades (intranets, gestión documental, trabajo en red, herramientas compartidas accesibles por múltiples usuarios, accesos diferenciados...)
- **Servicios a usuarios** (gestión de incidencias, accesos a contenidos diferenciados por permisos, gestión de espacios...)
- Herramientas de **comunicación digital** (mailings, boletines digitales, comunicaciones personalizadas a clientes o usuarios...)
- Herramientas de **Control de Calidad** de la empresa.
- **Herramientas web** (tiendas virtuales personalizadas, webs complejas con multitud de elementos y accesos, repositorios y buscadores, gestión de ventas online...)
- Otros tipos de servicios (**cualquier tipo de aplicación que se te ocurra** como gestión de inmuebles, comunidades de propietarios, turismo, mapas, formación, colegios, tiendas... y otra muchas que están por crear)

Una aplicación web es una aplicación o herramienta informática **accesible desde cualquier navegador**, bien sea a través de internet (lo habitual) o bien a través de una red local. A través del navegador se puede acceder a toda la funcionalidad y tener cualquiera de las soluciones enumeradas en el punto anterior.

Tiene unas grandes **ventajas**:

- La primera gran ventaja es que **no necesita ningún tipo de instalación**, ya que se accede a través de un navegador.
- Además, una aplicación web es **multiplataforma** y **multidispositivo**. Esto significa que nos podemos olvidar de que software tiene cada dispositivo que accede, y que puede acceder igualmente un ordenador, una Tablet, un smartphone...
- Y otra gran ventaja es que la **potencia** no está en el dispositivo que accede, por lo que, aunque no tengamos un superordenador la aplicación puede ser

muy potente, ya que el peso no lo soporta el equipo desde el que se accede sino el servidor donde está alojada.

- La aplicación puede estar en la **nube**, con lo que sería accesible para cualquier ordenador con acceso a internet (aunque también podría ser una aplicación local en una intranet)
- Y para finalizar, es muy adaptable, visualmente **intuitiva** y muy fácil de actualizar si fuera necesario.

Desventajas

- Habitualmente ofrecen menos funcionalidades que las aplicaciones de escritorio. Se debe a que las funcionalidades que se pueden realizar desde un navegador son más limitadas que las que se pueden realizar desde el sistema operativo. Pero cada vez los navegadores están más preparados para mejorar en este aspecto. La aparición de HTML 5 representa un hito en este sentido. Es posible añadir funcionalidades a estas aplicaciones gracias al uso de Aplicaciones de Internet Ricas.
- La disponibilidad depende de un tercero, el proveedor de la conexión a internet o el que provee el enlace entre el servidor de la aplicación y el cliente. Así que la disponibilidad del servicio está supeditada al proveedor.

Las aplicaciones web se ejecutan nativamente desde el navegador. Pero existen algunas aplicaciones que funcionan desde el navegador, pero además requieren la instalación de un software en el ordenador para poder utilizarse. Estas aplicaciones se denominan Aplicaciones de Internet Ricas. El motivo de usar este software adicional es que hay muchas funcionalidades que los navegadores no pueden ofrecer, y él enriquece a las aplicaciones web ofreciendo dichas funcionalidades. . Ejemplos de funcionalidades que pueden ofrecer los programas online gracias al uso de software instalado:

- Procesamiento de imágenes
- Captura de imágenes
- Uso de Webcam Captura de video

Algunos ejemplos concretos de Aplicaciones web

- Aplicación Web para **comunidades de propietarios**: Intranet, gestión documental, tablón de anuncios, gestión de incidencias en las comunidades, comunicación con los propietarios, repositorio documental, estadísticas para propietarios con análisis de resultados....
- Aplicación Web para **gestión de inmuebles**: Alta, baja y modificación de inmuebles, estadísticas y análisis de cambios, histórico de modificaciones, estadísticas de valores numéricos, acceso diferenciado por permisos, cálculos de cifras globales, exportación de resultados...
- Aplicación Web de **gestión de incidencias**: Alta de incidencias, asignación de técnicos, control de resoluciones, tiempos de atención, histórico de

modificaciones, gestión de diferentes tipos de contratos de clientes, gestión de clientes....

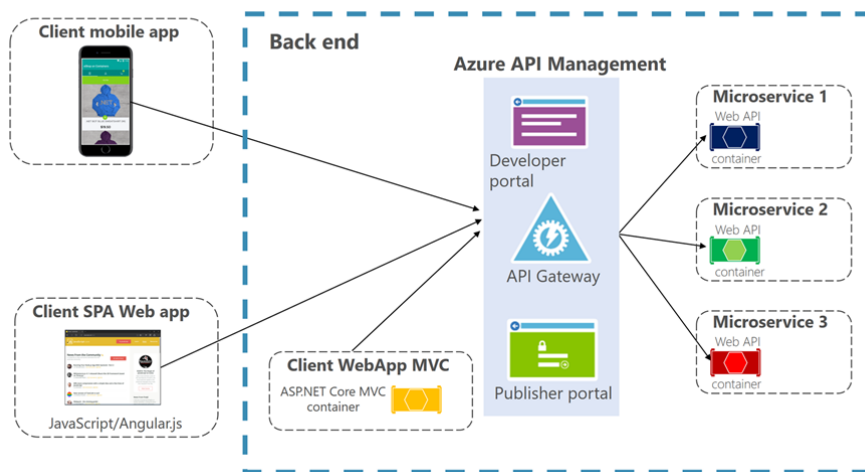
- Aplicación Web para la **gestión de ventas**: Tienda virtual con pago online, gestión automática de facturación, control interno de ventas, estadísticas por clientes, control de productos, gestión de descuentos por producto o por cliente, diferenciación de clientes...
- Aplicación Web para **fichar**: Aplicación para diferentes usuarios de una entidad para fichar desde diferentes lugares de trabajo, gestión de bajas y ausencias, estadísticas de horas de entrada, tiempos de trabajo....
- Aplicación Web **Turismo**: Gestión de mapas con diferentes tipos de puntos, reservas, espacios de información, repositorio de documentación turística, contacto....
- Aplicación Web Portal: Portal web complejo con campañas de firmas, recogida de donaciones, multiples elementos como emergencias, actualizaciones, noticias de diferentes tipos, comunicaciones digitales, recogida de datos personalizados....
- Aplicación Web **Comunicación**: Gestión de envíos de comunicaciones a clientes, gestión de usuarios automática con altas, bajas y modificaciones seguras por parte del propio usuario, diferentes tipos de envíos con plantillas

Modelo de arquitectura de servicios

La aplicación web tiene 3 partes:

- La **base de datos**, que es donde se almacenan los datos de la solución (contenidos, usuarios, permisos...)

Recommended API Gateway use with Azure API Management for production environments



- El código de la aplicación, es decir, la **aplicación en sí**, que se almacena en un servidor de aplicaciones (normalmente en la nube, pero puede estar alojado también en un servidor local)

- El usuario, que accede con cualquier dispositivo a través de un **navegador**. Aquí se incluye, dependiendo de la aplicación, tanto el administrador o gestor como el usuario final (en caso de que se ofrezca un servicio), que accederán a diferentes secciones según sus permisos (o incluso aplicaciones)

Modelo Cliente/Servidor

La **arquitectura cliente-servidor** es un modelo de aplicación distribuida en el que las tareas se reparten entre los proveedores de recursos o servicios, llamados servidores, y los demandantes, llamados clientes. Un cliente realiza peticiones a otro programa, el servidor, quien le da respuesta. Esta idea también se puede aplicar a programas que se ejecutan sobre una sola computadora, aunque es más ventajosa en un sistema operativo multiusuario distribuido a través de una red de computadoras.

Algunos ejemplos de aplicaciones computacionales que usen el modelo cliente-servidor son el Correo electrónico, un Servidor de impresión y la World Wide Web.

La red cliente-servidor es una red de comunicaciones en la cual los clientes están conectados a un servidor, en el que se centralizan los diversos recursos y aplicaciones con que se cuenta; y que los pone a disposición de los clientes cada vez que estos son solicitados. Esto significa que todas las gestiones que se realizan se concentran en el servidor, de manera que en él se disponen los requerimientos provenientes de los clientes que tienen prioridad, los archivos que son de uso público y los que son de uso restringido, los archivos que son de sólo lectura y los que, por el contrario, pueden ser modificados, etc. Este tipo de red puede utilizarse conjuntamente en caso de que se esté utilizando en una red mixta.

Características

En la arquitectura C/S el **remitente de una solicitud** es conocido como cliente. Sus características son:

- Es quien inicia solicitudes o peticiones, tienen por tanto un papel activo en la comunicación (dispositivo **maestro** o **amo**).
- Espera y recibe las respuestas del servidor.
- Por lo general, puede conectarse a varios servidores a la vez.
- Normalmente interactúa directamente con los usuarios finales mediante una interfaz gráfica de usuario.

Al **receptor de la solicitud** enviada por el cliente se conoce como servidor. Sus características son:

- Al iniciarse esperan a que lleguen las solicitudes de los clientes, desempeñan entonces un papel pasivo en la comunicación (dispositivo **esclavo**).
- Tras la recepción de una solicitud, la procesan y luego envían la respuesta al cliente.
- Por lo general, acepta las conexiones de un gran número de clientes (en ciertos casos el número máximo de peticiones puede estar limitado).

En la arquitectura C/S sus características generales son:

- El Cliente y el Servidor pueden actuar como una sola entidad y también pueden actuar como entidades separadas, realizando actividades o tareas independientes.
- Las funciones de Cliente y Servidor pueden estar en plataformas separadas, o en la misma plataforma.
- Cada plataforma puede ser escalable independientemente. Los cambios realizados en las plataformas de los Clientes o de los Servidores, ya sean por actualización o por reemplazo tecnológico, se realizan de una manera transparente para el usuario final.
- La interrelación entre el hardware y el software están basados en una infraestructura poderosa, de tal forma que el acceso a los recursos de la red no muestra la complejidad de los diferentes tipos de formatos de datos y de los protocolos.
- Su representación típica es un centro de trabajo (PC), en donde el usuario dispone de sus propias aplicaciones de oficina y sus propias bases de datos, sin dependencia directa del sistema central de información de la organización.

Componentes básicos del modelo Cliente-Servidor

En esta aproximación, y con el objetivo de definir y delimitar el modelo de referencia de una arquitectura Cliente/Servidor, se identifica 5 componentes que permitan articular dicha arquitectura, considerando que toda aplicación de un sistema de información está caracterizada por siguiente:

- Presentación/Captación de la información.
- Procesos.
- Almacenamiento de la información.
- Puestos de trabajo
- Comunicaciones.

Ventajas

- Centralización del control: los accesos, recursos y la integridad de los datos son controlados por el servidor de forma que un programa cliente defectuoso o no autorizado no pueda dañar el sistema. Esta centralización también facilita la tarea de poner al día datos u otros recursos (mejor que en las redes P2P)..
- Escalabilidad: se puede aumentar la capacidad de clientes y servidores por separado. Cualquier elemento puede ser aumentado (o mejorado) en cualquier momento, o se pueden añadir nuevos nodos a la red (clientes y/o servidores).
- Fácil mantenimiento: al estar distribuidas las funciones y responsabilidades entre varios ordenadores independientes, es posible reemplazar, reparar,

actualizar, o incluso trasladar un servidor, mientras que sus clientes no se verán afectados por ese cambio (o se afectarán mínimamente). Esta independencia de los cambios también se conoce como encapsulación.

- Existen tecnologías, suficientemente desarrolladas, diseñadas para el paradigma de C/S que aseguran la seguridad en las transacciones, la amigabilidad de la interfaz, y la facilidad de empleo.

Desventajas

- La congestión del tráfico ha sido siempre un problema en el paradigma de C/S. Cuando una gran cantidad de clientes envían peticiones simultáneas al mismo servidor, puede ser que cause muchos problemas para éste (a mayor número de clientes, más problemas para el servidor). Al contrario, en las redes P2P como cada nodo en la red hace también de servidor, cuanto más nodos hay, mejor es el ancho de banda que se tiene.
- El paradigma de C/S clásico no tiene la robustez de una red P2P. Cuando un servidor está *caído*, las peticiones de los clientes no pueden ser satisfechas. En la mayor parte de redes P2P, los recursos están generalmente distribuidos en varios nodos de la red. Aunque algunos salgan o abandonen la descarga; otros pueden todavía acabar de descargar consiguiendo datos del resto de los nodos en la red.
- El software y el hardware de un servidor son generalmente muy determinantes. Un hardware regular de un ordenador personal puede no poder servir a cierta cantidad de clientes. Normalmente se necesita software y hardware específico, sobre todo en el lado del servidor, para satisfacer el trabajo. Por supuesto, esto aumentará el coste.
- El cliente no dispone de los recursos que puedan existir en el servidor. Por ejemplo, si la aplicación es una Web, no podemos escribir en el disco duro del cliente o imprimir directamente sobre las impresoras sin sacar antes la ventana previa de impresión de los navegadores.

Instalación y configuración de servidores Web

Por lo general, si creamos una página web solo con lenguaje frontend, por ejemplo en HTML y CSS, el propio navegador es capaz de interpretarla correctamente, por lo que no necesitamos disponer de ningún servidor adicional instalado en el sistema. Sin embargo, si vamos a hacer uso de lenguajes backend, por ejemplo en PHP, debemos instalar los servidores necesarios en nuestro ordenador (o servidor) de manera que estos sean capaces de interpretar el lenguaje y nos permiten ver el resultado final a través del navegador que, sin dicho servidor, el navegador no interpretaría.

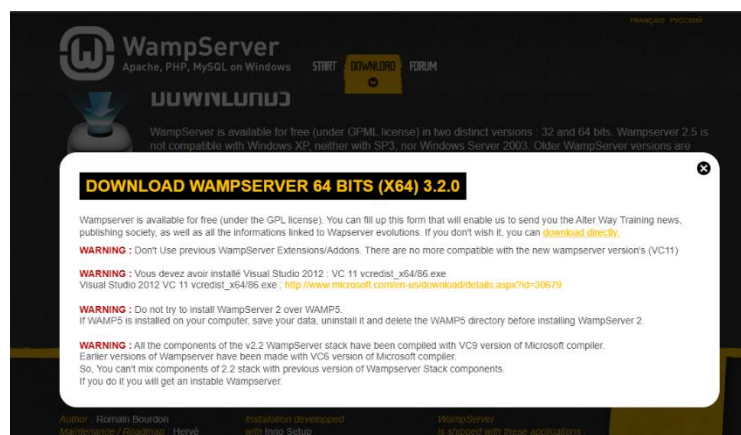
Wamp es un paquete de servidores web para Windows totalmente gratuito y de código abierto que nos va a permitir instalar desde un único paquete un servidor Apache con PHP y un servidor MySQL para poder ejecutar en local cualquier página web que desarrollemos por nuestra cuenta sin tener que subir los archivos a un servidor web dedicado conectado a la nube.

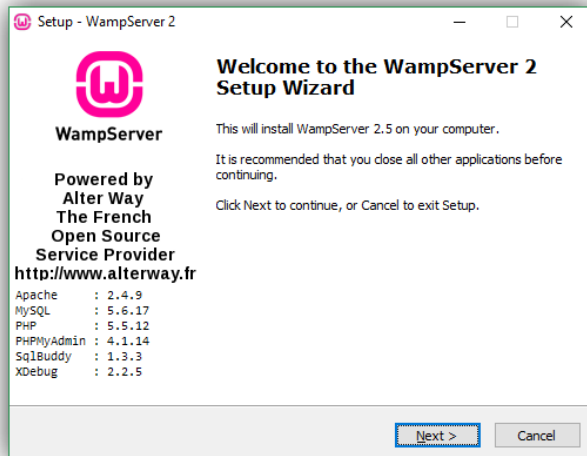


Descargar e instalar Wamp en un equipo Windows

Lo primero que debemos hacer es descargar el programa [desde su página web principal](#). El programa es totalmente gratuito y está libre de software publicitario y no deseado.

En su página web vamos a poder elegir si queremos descargar la versión de **32 bits o la de 64 bits**. En nuestro caso vamos a trabajar con la versión de 64 bits, ya que tanto nuestro procesador como nuestro sistema operativo lo permiten. Pulsamos sobre la versión deseada de **Wamp Server** y, en la ventana que nos aparece, pulsamos sobre el enlace «**download directly**».

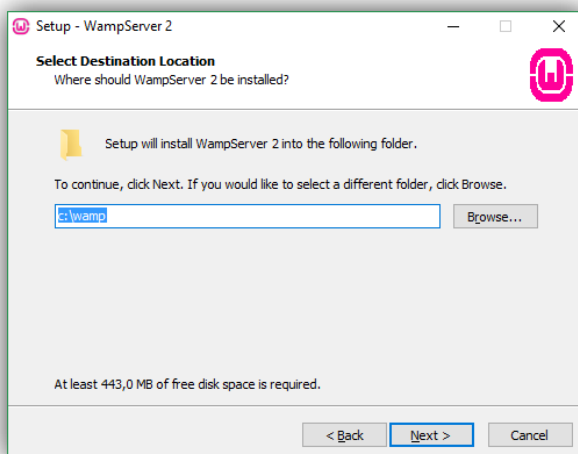
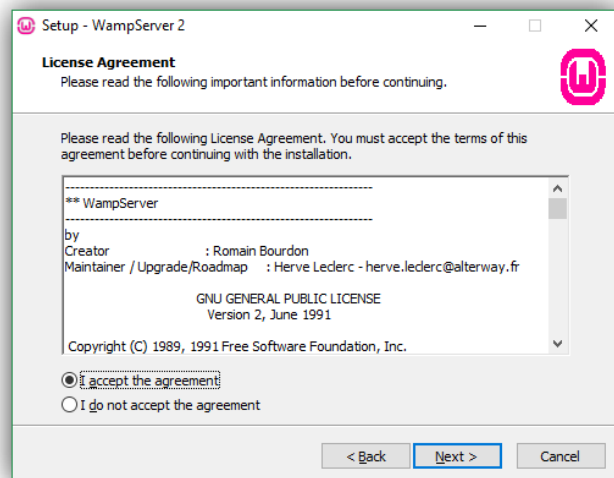




El paquete, de **poco más de 500 megas**, empezará a descargarse a nuestro ordenador. Una vez lo tengamos ya podemos comenzar la instalación.

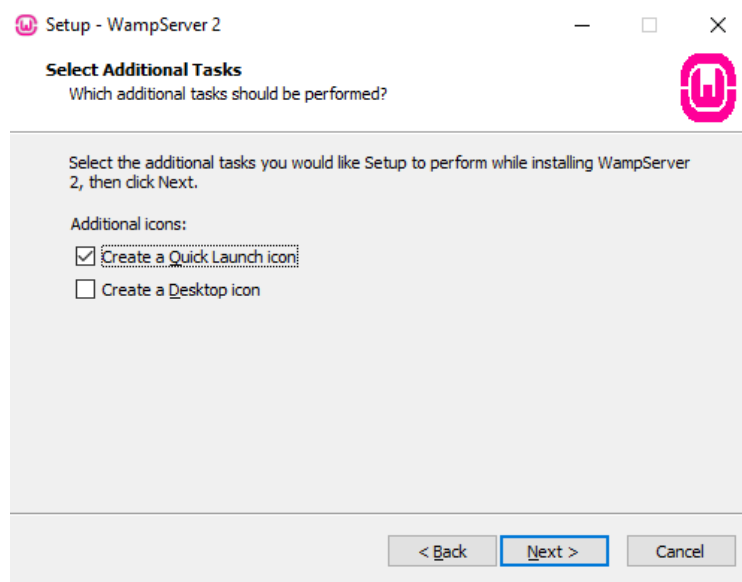
Ejecutamos el instalador que hemos descargado en el paso anterior y podremos ver la primera ventana del sencillo asistente, el cual no tiene demasiada complicación.

Seguimos con este asistente y a continuación debemos leer y aceptar los términos de licencia. **Wamp es software libre**, por lo que la licencia que utiliza el GNU

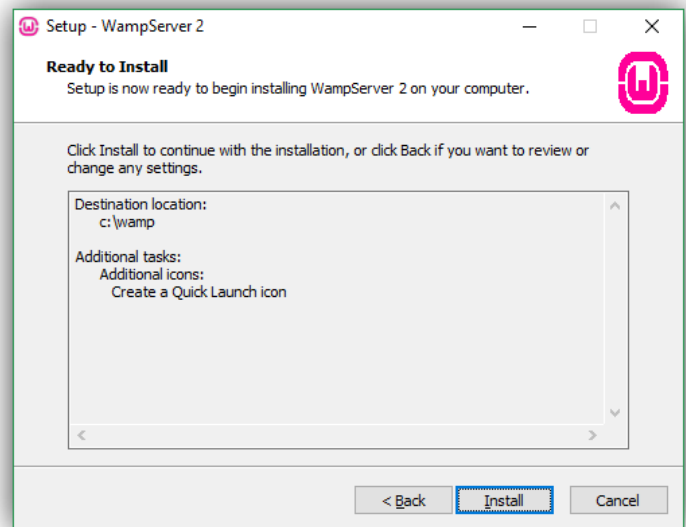


A continuación, elegimos la ruta de instalación. Por defecto se instalará en raíz de c:\, una buena ruta y de fácil acceso, por lo que si no tenemos ninguna necesidad específica podemos dejarlo así y continuar.

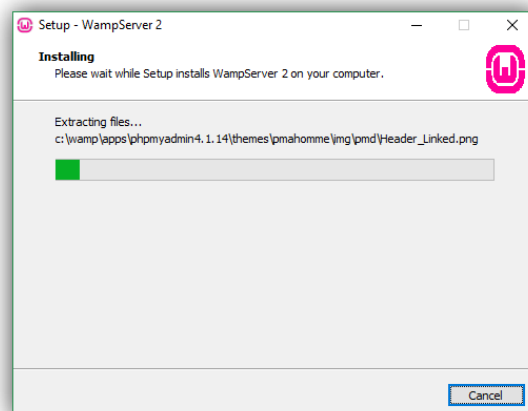
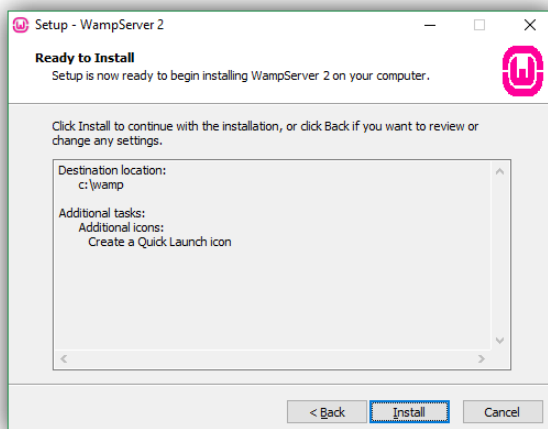
A continuación, podremos elegir si queremos crear o no iconos en el escritorio o en el menú inicio.



Seguimos con el asistente y llegamos caso al final. Podremos ver un resumen de todas estas opciones.

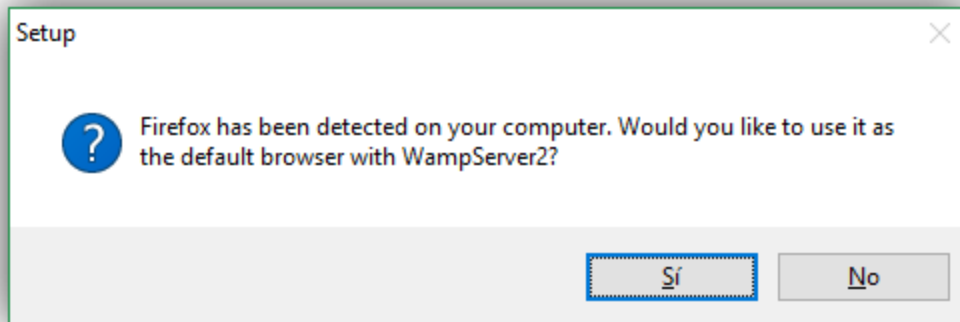


Si todo está correcto podemos comenzar con la instalación. Pulsamos sobre «Install» y comenzará la copia de archivos.



Este proceso puede tardar algunos minutos, por lo que debemos esperar a que finalice correctamente. Durante la copia de archivos recibiremos dos avisos:

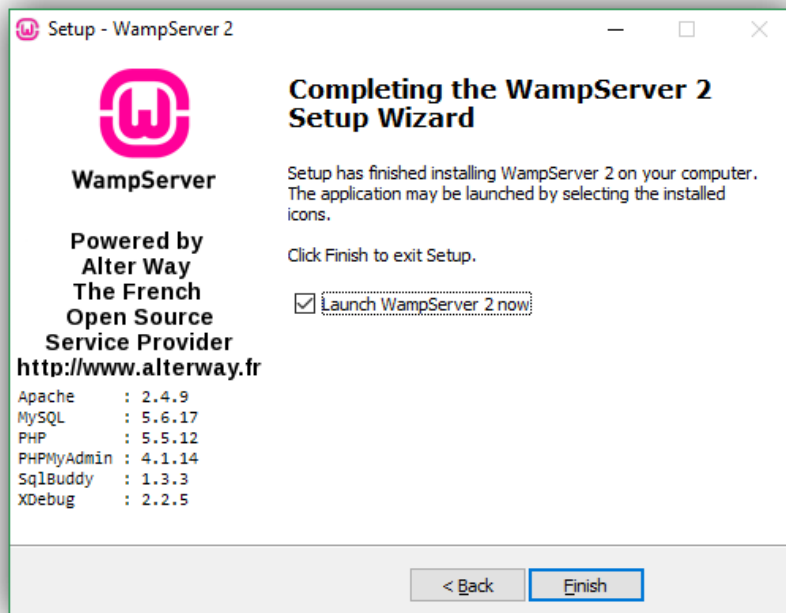
El primero de ellos nos preguntará por nuestro navegador por defecto. Si queremos que el navegador por defecto sea el que utilice Wamp para abrir las webs que alojemos en él, aceptamos dicha ventana.



El segundo de ellos nos pide rellenar una información básica de cara al servidor de correo SMTP que habilita Wamp. Podemos dejar perfectamente los valores por defecto ya que, como vamos a trabajar en localhost, no tendremos problemas.

Listo. Wamp está instalado en nuestro ordenador y ya podemos empezar a utilizarlo.

Consejos básicos sobre el funcionamiento



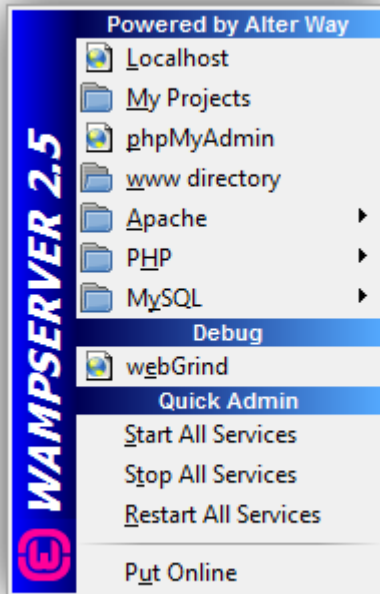
Lejos de entrar en una configuración avanzada del servidor, a continuación, os vamos a explicar los principales puntos a tener en cuenta.

El primero de ellos es el icono del servidor. Junto al reloj, en la barra de tareas, podremos ver un icono con forma de W que puede tener 3 colores diferentes, según el estado de este:

- **Rojo:** todos los servidores están detenidos o no pueden arrancar.

- **Naranja:** algún servidor está detenido o no ha podido arrancar, pero los demás funcionan.
- **Verde:** todo funciona correctamente.

Si pulsamos sobre dicho icono vamos a poder abrir un pequeño menú de uso, desde donde podremos comprobar todos los servidores e incluso forzar a que los servicios de Wamp se reinicien, detengan o inicien de nuevo.



Para finalizar, suponiendo que hemos instalado el programa en C:\wamp, la ruta por defecto, las páginas web que creemos debemos almacenarlas en la ruta **c:\wamp\www** para que el servidor pueda detectarlas, cargarlas e interpretarlas adecuadamente.

Cómo solucionar los principales problemas

Para funcionar correctamente, Wamp necesita conectarse a Internet, por lo que si alguna otra aplicación está utilizando o bloqueando el puerto 80 es muy probable que el icono no termine de aparecer de color verde.

Una de las principales causas de esto es el servidor web de Microsoft instalado por defecto en Windows, **IIS**. En caso de que tengamos este servidor web habilitado en el sistema debemos

desinstalarlo, o simplemente detenerlo, para que no interfiera con el funcionamiento de Wamp.

De todas formas, cada ordenador es un mundo, por lo que es posible que aparezcan errores y problemas de todo tipo en función de la configuración que tengamos y los programas previamente instalados.

Si el problema no es de IIS, es posible que debemos revisar si algún otro servidor está haciendo uso del puerto 80 o si tenemos alguna aplicación de seguridad, por ejemplo, un Firewall, que está bloqueando nuestras conexiones con el exterior.

Modelo Relacional

Modelo de Datos

El modelo relacional se basa en el concepto matemático de relación, que gráficamente se representa mediante una tabla. Es decir, una relación es una tabla, con columnas y filas. Un SGBD sólo necesita que el usuario pueda percibir la base de datos como un conjunto de tablas.

Frecuentemente una relación se conceptualiza de una manera más fácil de imaginar, esto es, pensando en cada relación como si fuese una tabla que está compuesta por registros (cada fila de la tabla sería un registro o tupla), y columnas (también llamadas campos).

Un atributo es el nombre de una columna de una relación. En este modelo, las relaciones se utilizan para almacenar información sobre los objetos que se representan en la base de datos. Una relación se representa gráficamente como una tabla bidimensional en la que las filas corresponden a registros individuales y las columnas corresponden a los campos o atributos de esos registros. Los atributos pueden aparecer en la relación en cualquier orden. El grado de una relación es el número de atributos que contiene. En términos más sencillos, es el número de campos que tiene una determinada tabla.

Un dominio es el conjunto de valores legales de uno o varios atributos. Los dominios constituyen una poderosa característica del modelo relacional. Cada atributo de una base de datos relacional se define sobre un dominio, pudiendo haber varios atributos definidos sobre el mismo dominio. El concepto de dominio es importante porque permite que el usuario defina, en un lugar común, el significado y la fuente de los valores que los atributos pueden tomar.

Esto hace que haya más información disponible para el sistema cuando éste va a ejecutar una operación relacional, de modo que las operaciones que son semánticamente incorrectas se pueden evitar. Por ejemplo, no tiene sentido comparar el nombre de una calle con un número de teléfono, aunque los dos atributos sean cadenas de caracteres. Sin embargo, el importe mensual del alquiler de un inmueble no estará definido sobre el mismo dominio que el número de meses que dura el alquiler. Pero sí tiene sentido multiplicar los valores de ambos dominios para averiguar el importe total al que asciende el alquiler.

En resumen, el dominio es el conjunto de valores que puede asumir un determinado atributo (campo). Una tupla es una fila de una relación. Los elementos de una relación son las tuplas o filas de la tabla. Las tuplas de una relación no siguen ningún orden. En términos más sencillos, las tuplas son los diferentes registros de una determinada tabla.



A continuación, presentamos el siguiente ejemplo. Nombre de la tabla: Representantes

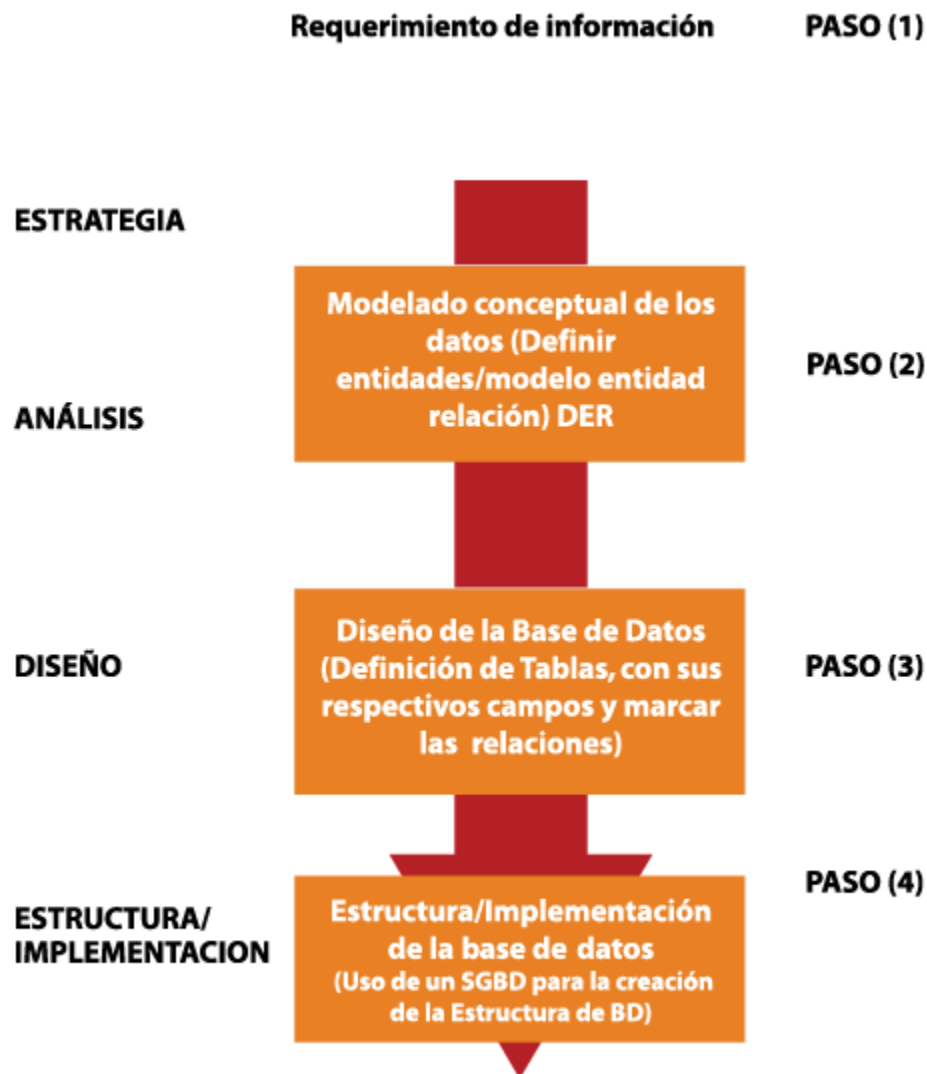


Atributos. Campos: Nombre, Nacionalidad, Institución. Por lo tanto, como se explicó anteriormente sobre el grado de relación, en el ejemplo presentado identificamos una relación de grado 3, ya que son 3 atributos o campos y tiene 4 tuplas o registros. Podemos observar además que la tabla (relación) debe reunir un conjunto de requisitos:

- No puede haber filas duplicadas, es decir, todas las tuplas tienen que ser distintas.
- El orden de las filas es irrelevante.
- La tabla es plana, es decir, en el cruce de una fila y una columna sólo puede haber un valor (no se admiten atributos multivaluados).

Para que pueda cumplirse el primer requisito, debe ser posible definir lo que se conoce como clave primaria. Se llama clave primaria a un campo o a una combinación de campos que identifica de forma única a cada fila de una tabla. Una clave primaria comprende de esta manera una columna o conjunto de columnas. No puede haber dos filas en una tabla que tengan la misma clave primaria. En otras palabras, una clave primaria debe identificar unívocamente a todas las posibles filas de una tabla y no solo a las filas que se encuentran en un momento determinado.

Complementando el concepto de claves, están las llamadas claves foráneas (foreign key) que identifican una columna o grupo de columnas en una tabla (tabla hija o referendo) que se refiere a una columna o grupo de columnas en otra tabla (tabla maestra o referenciada). Las columnas en la tabla hija deben ser la clave primaria u otra clave candidata en la tabla referenciada. De esta manera es posible enlazar las diferentes tablas que definan una base de datos.



Primer paso. Pensar estratégicamente el requerimiento de información. La primera etapa consiste en resistir la tentación de abrir la aplicación y empezar a dar clic y teclear. Esto solo puede tener éxito en aquellos casos en que la problemática a resolver sea muy sencilla. Pensar requiere que seamos estratégicos: frente al requerimiento de información, debemos encontrar la forma más eficiente de lograrla.

Segundo paso. Modelado conceptual. Consta en definir el conjunto de entidades que intervienen en el problema a resolver y en cómo éstas se relacionan entre sí. Para ello, usualmente se utiliza un diagrama de entidad / relación. Debemos

considerar que, frecuentemente, las entidades que definamos guardan estrecha relación con las tablas que contendrá la base de datos que estamos diseñando. El diagrama de entidad-relación (DER) describe los datos almacenados en una base de datos y sus relaciones. Veamos el siguiente diagrama de un sistema de facturación sencillo:

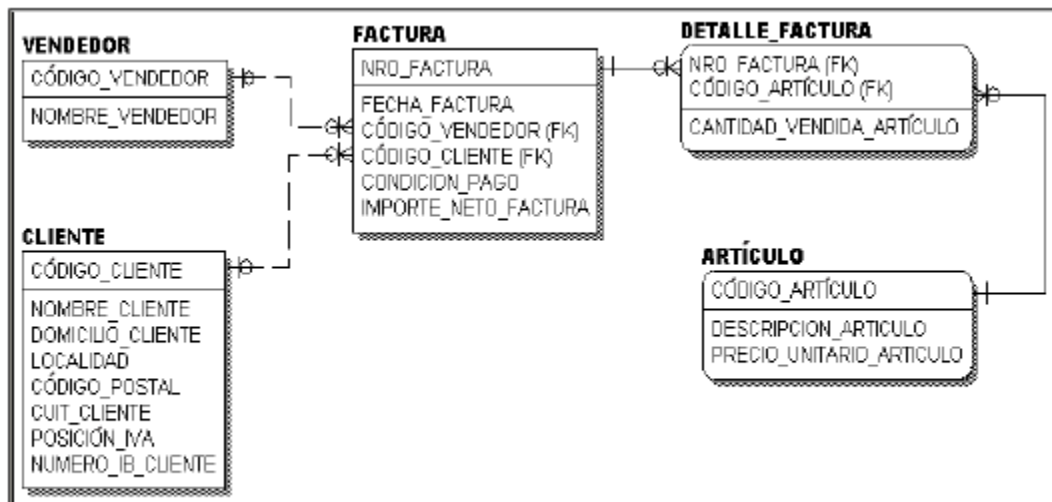


Diagrama de E/R. Sistema de facturación

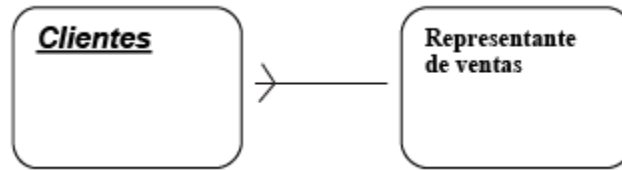
Cada entidad se representa por un rectángulo y tiene un nombre. Por ejemplo, la entidad vendedor tiene dos atributos o campos, de los cuales código_vendedor es su clave primaria (primary key). Podemos advertir que en la entidad factura existe el mismo campo pero, en ese caso, es clave foránea (foreign key).

Las líneas que enlazan las tablas indican las relaciones existentes entre ellas. Estas relaciones entre las tablas pueden ser de diferentes tipos o cardinalidades:

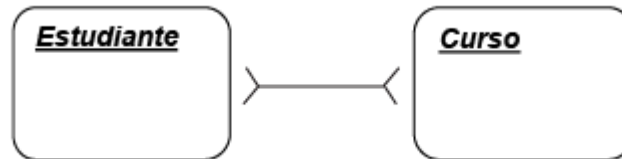
- Uno a Uno (1:1) Ej: Existe una relación entre el Procesador y la Tarjeta madre. Cada PC debe tener una y solo una Tarjeta Madre Local. Cada Tarjeta Madre puede ser incorporada dentro de una y sola una PC.



- Uno a Muchos (1 a M ó 1:M) Ej: Existe una relación 1:M entre los clientes y un representante de ventas. Cada Cliente puede ser visitado por uno y solo un representante de ventas. Cada Representante de Ventas puede ser asignado a visitar a uno o más clientes.



Mucho a Muchos (M:M) Ej: Existe una relación M:M entre un estudiante y un curso. Cada estudiante puede estar inscrito en uno ó más cursos. Cada Curso puede estar compuesto por uno o más cursos.



Tercer paso. El diseño de la base de datos. La planificación de la estructura de la base de datos, en particular de las tablas, es vital para la gestión efectiva de ellas. El diseño de la estructura de una tabla consiste en una descripción de cada uno de los campos que componen el registro y los tipos de datos que contendrá cada uno de esos campos, como así también su tamaño o longitud. Por ejemplo, dada la entidad-persona para definir la tabla que almacenará información sobre ella, debemos determinar los diferentes atributos que definen a persona, y el tipo de datos que corresponde a cada uno. Veámoslo:

Nombre del atributo	DNI	Nombre	Dirección	Sexo
Tipo de atributo	Numérico	Texto	Texto	Texto
Tamaño Longitud	10	35	50	1

Ej.:

PERSONA

Campo	Tipo	Tamaño	Otros
DNI	Numérico	10	ID
Nombre	Texto	35	
Dirección	Texto	50	
Sexo	Texto	1	

Cuarto paso. Utilizar un gestor de BD para la creación, manipulación, y consulta de la BD. Una vez diseñada la BD, este último paso es muy sencillo y consiste en utilizar un gestor de base de datos. En nuestro caso, podrá ser MS Access o bien Open Office Base, y procederemos a: crear las diferentes tablas de

la BD; crear en cada tabla los campos, de acuerdo al tipo de datos definido en el diseño; establecer o definir los campos clave; en función de las claves definidas, ya sean primarias o foráneas, se establecerán las relaciones correspondientes; cargar las tablas con los registros que correspondan.

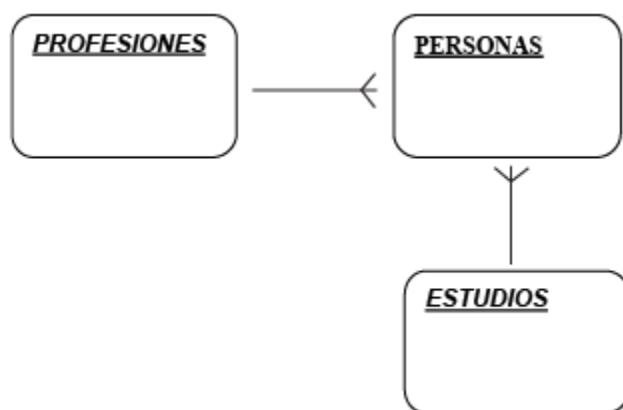
A continuación, plantearemos un caso práctico para visualizar las fases requeridas en la creación de un SGBD. Le recomendamos tener encendida su computadora, escribir aquellas dudas o comentarios que surgen durante la lectura y realizar todas las consultas necesarias a su profesor-tutor a través del aula virtual.

Diseño de Bases de Datos

Planteamiento del problema: Se necesita crear una base de datos que permita gestionar un conjunto de personas y la formación que poseen (estudios y profesión).

Paso 1. Como dijimos anteriormente, se trata de pensar sobre nuestras necesidades de información. En esta oportunidad, requerimos información de las personas y sus estudios. Para ello deberemos tener en cuenta que una persona puede poseer diferentes niveles de estudio y profesiones, por ejemplo: habrá quienes, por su nivel de estudios, no tengan profesión; otros que son médicos, abogados, ingenieros. Incluso habrá algunas personas que tengan más de una profesión: por ejemplo: abogado y contador. A los efectos de simplificar el modelo, en el nuestro excluirémos esta posibilidad, y diremos que las personas sólo tienen una profesión.

Paso 2. Este modelo consiste en definir el conjunto de entidades que intervienen en el problema a resolver. Frecuentemente se utilizan herramientas como el Diagrama de Entidad Relación (DER). Cada PC debe tener una y solo una Tarjeta Madre Local.



En el diagrama presentando arriba, observamos que la relación es 1:M (uno a muchos), ya que habrá grupos de personas que tienen estudios secundarios, otros grupos que tienen estudios universitarios, etc. Dominio de estudios:

- Sin estudios
- Primario
- Secundario
- Terciarios
- Universitarios

Una persona puede tener profesiones, pero recordemos las restricciones de nuestro modelo, una persona solo tiene una profesión. La relación, entonces, es 1:M, ya que por cada profesión definida podrán existir más de una persona que tenga la misma profesión. Dominio de profesiones:

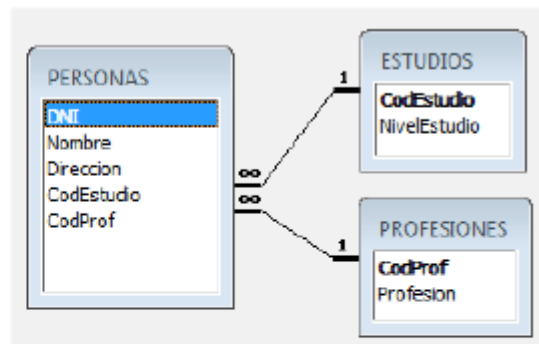
- No profesional
- Médico
- Abogado
- Biólogo
- Físico
-
- Astrónomo

Paso 3 y 4. Dada la sencillez de nuestro ejemplo, exponemos los dos últimos pasos en conjunto. Solo faltaría cargar los registros correspondientes en cada tabla.

	Nombre del campo	Tipo de datos
?	DNI	Número
	Nombre	Texto
	Direccion	Texto
	CodEstudio	Número
	CodProf	Número

	Nombre del campo	Tipo de datos
?	CodEstudio	Autonumérico
	NivelEstudio	Texto

	Nombre del campo	Tipo de datos
?	CodProf	Autonumérico
	Profesion	Texto



Definición de tablas
y establecimiento de
relaciones entre las
mismas

Para sistematizar

Lo invitamos a sistematizar en una base de datos aquella información de su lugar de trabajo o su biblioteca personal y compartir esta experiencia con sus compañeros.

En primer lugar, tendrá que diseñar una base que le permita almacenar los datos relevantes. Por ejemplo, si se tratase de una base de datos de libros, Ud. tendría que identificar el título de sus libros, datos del autor, cantidad de páginas, editorial, año de publicación, temática del libro, etc. Considere que tanto las editoriales como los autores publican más de un libro, por lo que es deseable que la información de editoriales y autores se mantenga en tablas separadas.

Realice el diseño correspondiente graficando el DER (Diagrama de Entidad/Relación). Una vez terminado, envíeselo a su tutor.

Finalmente, comparta con sus compañeros y tutor su experiencia: qué ámbito de información seleccionó, cómo elaboró su base de datos, qué dificultades encontró y cómo lo resolvió, así como la estructura que finalmente dio a su sistema.

Introducción a PHP

¿Qué es PHP?

PHP (acrónimo recursivo de *PHP: Hypertext Preprocessor*) es un lenguaje de código abierto muy popular especialmente adecuado para el desarrollo web y que puede ser incrustado en HTML.

Bien, pero ¿qué significa realmente? Un ejemplo nos aclarará las cosas:
Ejemplo #1 Un ejemplo introductorio

```
<!DOCTYPE html>
<html>
  <head>
    <title>Ejemplo</title>
  </head>
  <body>

    <?php
      echo "¡Hola, soy un script de PHP!";
    ?>

  </body>
</html>
```

En lugar de usar muchos comandos para mostrar HTML (como en C o en Perl), las páginas de PHP contienen HTML con código incrustado que hace "algo" (en este caso, mostrar "¡Hola, soy un script de PHP!"). El código de PHP está encerrado entre

las [etiquetas especiales de comienzo y final <?php y ?>](#) que permiten entrar y salir del "modo PHP".

Lo que distingue a PHP de algo del lado del cliente como Javascript es que el código es ejecutado en el servidor, generando HTML y enviándolo al cliente. El cliente recibirá el resultado de ejecutar el script, aunque no se sabrá el código subyacente que era. El servidor web puede ser configurado incluso para que procese todos los ficheros HTML con PHP, por lo que no hay manera de que los usuarios puedan saber qué se tiene debajo de la manga.

Lo mejor de utilizar PHP es su extrema simplicidad para el principiante, pero a su vez ofrece muchas características avanzadas para los programadores profesionales. No sienta miedo de leer la larga lista de características de PHP. En unas pocas horas podrá empezar a escribir sus primeros scripts.

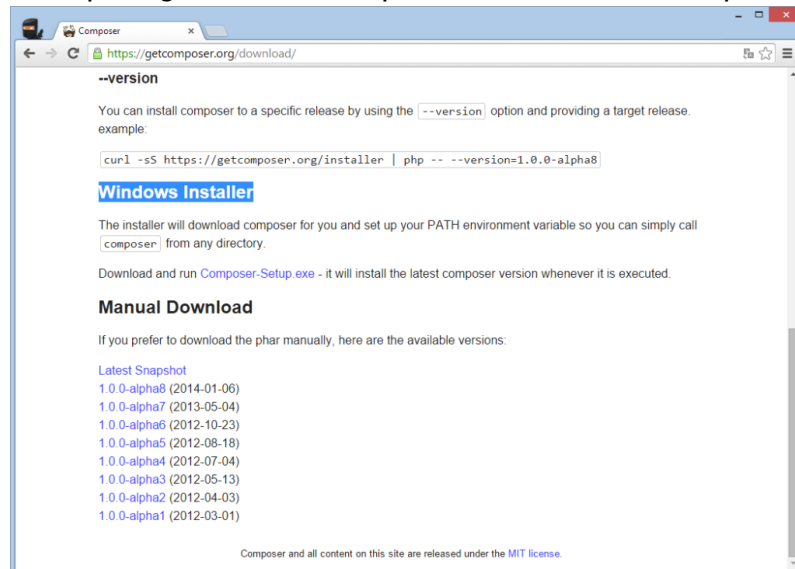
Aunque el desarrollo de PHP está centrado en la programación de scripts del lado del servidor, se puede utilizar para muchas otras cosas. Siga leyendo y descubra más en la sección [¿Qué puede hacer PHP?](#), o vaya directo al [tutorial introductorio](#) si solamente está interesado en programación web.

Instalación y configuración de PHP sobre servidor Web

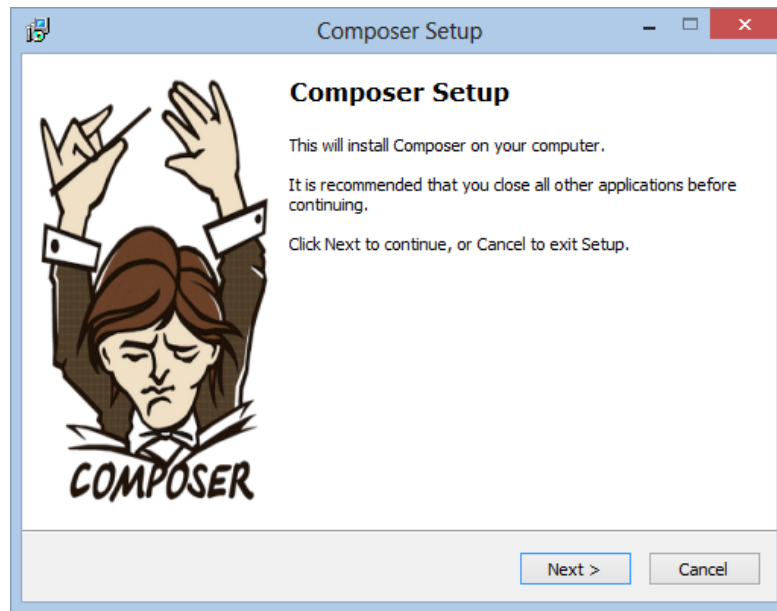
Instalar Composer

Composer es una herramienta para gestionar las dependencias en PHP. Te permite declarar las librerías de las cuales tu proyecto depende o necesita y las instala en el proyecto por ti, si deseas saber más acerca de Composer lee el siguiente [post](#).

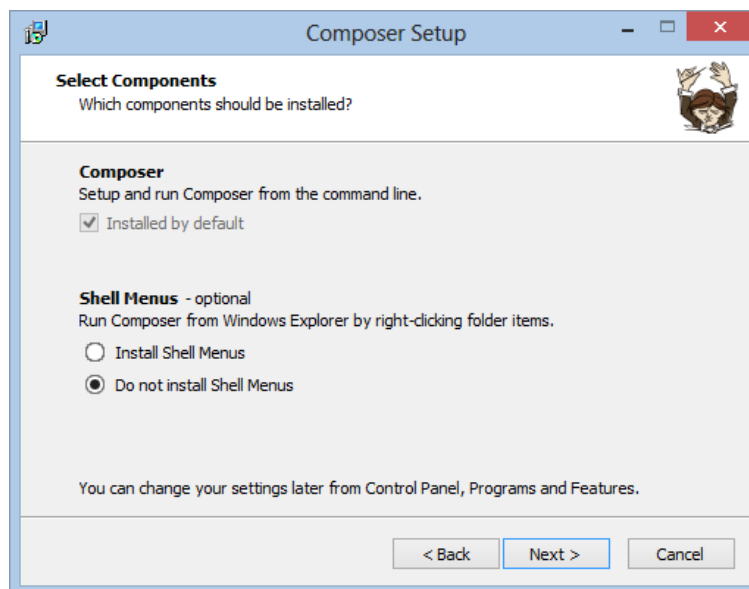
Para instalar Composer en Windows debemos descargarlo de su [página oficial](#) y en la sección Windows Installer, haz click en Composer-Setup.exe.



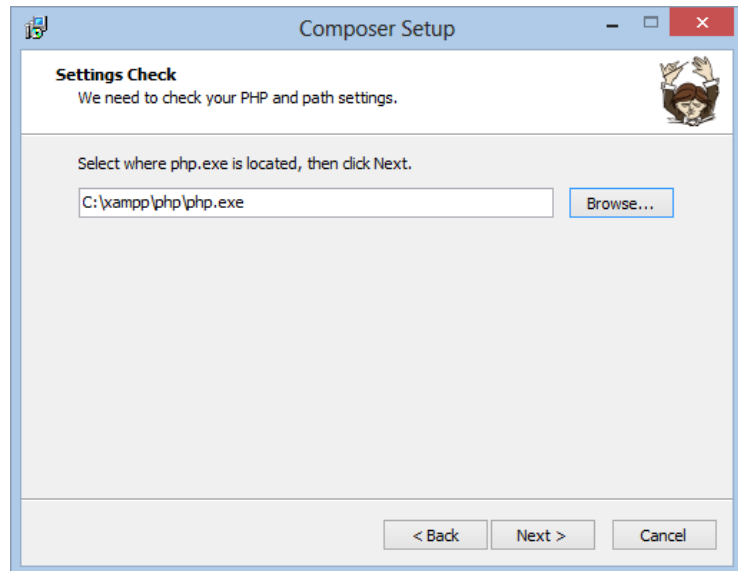
Una vez que la descarga finalice, ejecuta el instalador y haz click en Next.



Si quieres administrar tus proyectos mediante el Explorador de Windows puedes seleccionar la opción «Install Shell Menus» aunque lo recomendable es la usar la línea de comandos.

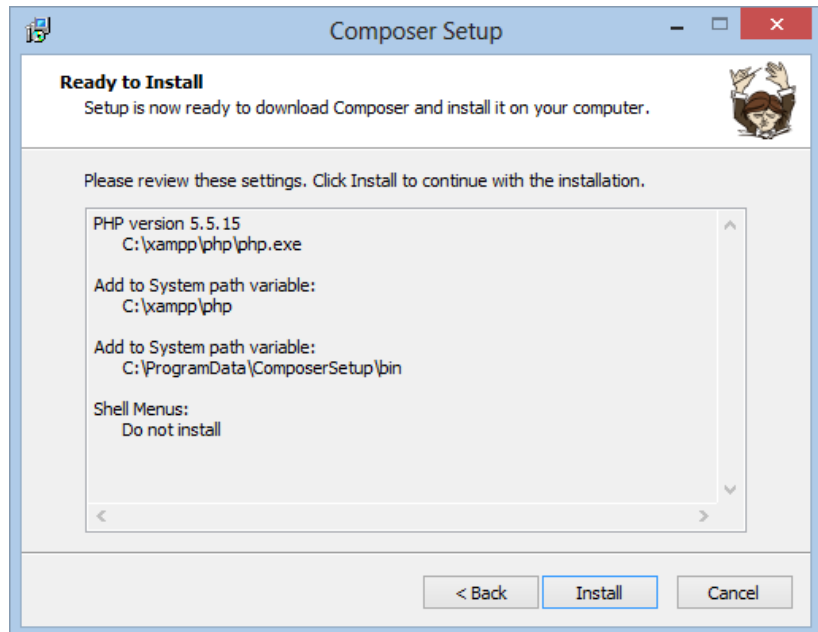


A continuación, nos pide que indiquemos la ruta del ejecutable de PHP, en mi caso como estoy trabajando con XAMPP el ejecutable de PHP se encuentra en la ruta C:\xampp\php\ (si usas WAMPP la ruta es C:\wamp\bin\php\php5.5.12) y seleccionas php.exe, luego click en Next.



En este punto el instalador de Composer nos muestra la configuración de la instalación, simplemente le damos click a Install.

Una vez esté todo instalado, aparecerán otras donde simplemente debes hacer click en Next, y posteriormente en Finalizar; después de tantos Next, Next típicos de Windows el instalador de Composer habrá puesto en nuestro PATH global la ruta de la

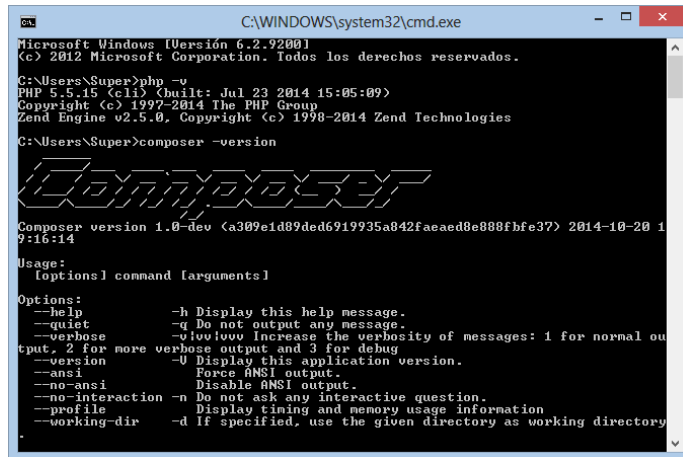


carpeta PHP y su propia carpeta Composer. Esto nos permite trabajar desde consola escribiendo sólo php o composer sin necesidad de indicar la ruta del ejecutable. Para ver que todo está en orden vamos a realizar dos pequeñas pruebas, así que es momento de abrir la consola, y teclear:

```
1 php -v (tecla Enter)
2 composer -version (tecla Enter)
```

Esto debería devolver la versión de cada uno, como se ve en la siguiente imagen:

Con esto ya tenemos Composer instalado y funcionando en Windows, ahora solo nos queda instalar Laravel, veamos cómo hacer esto posible.



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Versión 6.2.9200]
(c) 2012 Microsoft Corporation. Todos los derechos reservados.

C:\Users\Super>php -v
PHP 5.5.15 (cli) (built: Jul 23 2014 15:05:09)
Copyright (c) 1997-2014 The PHP Group
Zend Engine v2.5.0, Copyright (c) 1998-2014 Zend Technologies

C:\Users\Super>composer -version

Composer version 1.0-dev (a309e1d89ded6919935a842faeae8e888fbfc37) 2014-10-20 19:16:14

Usage:
  [options] command [arguments]

Options:
  --help           -h Display this help message.
  --quiet          -q Do not output any message.
  --verbose        -v|vv|vvv Increase the verbosity of messages: 1 for normal output, 2 for more verbose output and 3 for debug
  --version        -V Display this application version.
  --ansi           -A Force ANSI output.
  --no-ansi        -n Disable ANSI output.
  --no-interaction -n Do not ask any interactive question.
  --profile        -p Display timing and memory usage information
  --working-dir    -d If specified, use the given directory as working directory
```

Creando proyecto en Laravel

Existen dos formas de crear un proyecto con Laravel, la primera es descargando el archivo master desde su [repositorio oficial de GitHub](#) y la otra es usando Composer desde la consola que es precisamente lo que haremos en esta ocasión.

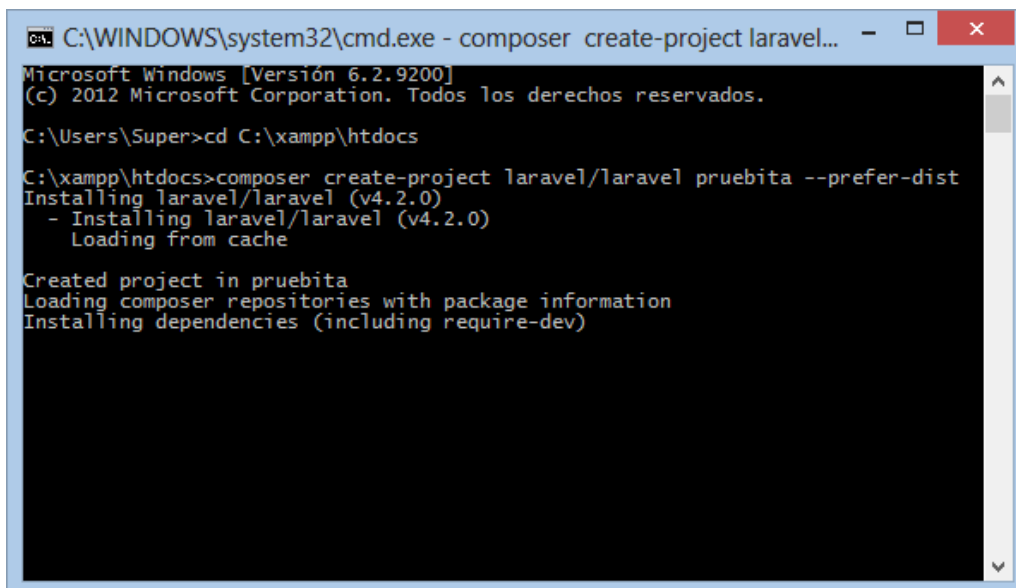
Desde la consola, dirígete al directorio donde guardas tus proyectos web (si usas XAMPP la ruta es C:\xampp\htdocs para WAMP es C:\wamp\www), y teclea lo siguiente:

```
1 cd C:\xampp\htdocs
```

Ahora crearemos el proyecto laravel escribiendo las siguientes palabras mágicas:

```
1 composer create-project laravel/laravel nombre_del_proyecto --prefer-dist
```

En mi caso en un arranque de creatividad llamaré a mi proyecto “pruebita”



```
C:\WINDOWS\system32\cmd.exe - composer create-project laravel...
Microsoft Windows [Versión 6.2.9200]
(c) 2012 Microsoft Corporation. Todos los derechos reservados.

C:\Users\Super>cd C:\xampp\htdocs

C:\xampp\htdocs>composer create-project laravel/laravel pruebita --prefer-dist
Installing laravel/laravel (v4.2.0)
- Installing laravel/laravel (v4.2.0)
  Loading from cache

Created project in pruebita
Loading composer repositories with package information
Installing dependencies (including require-dev)
```

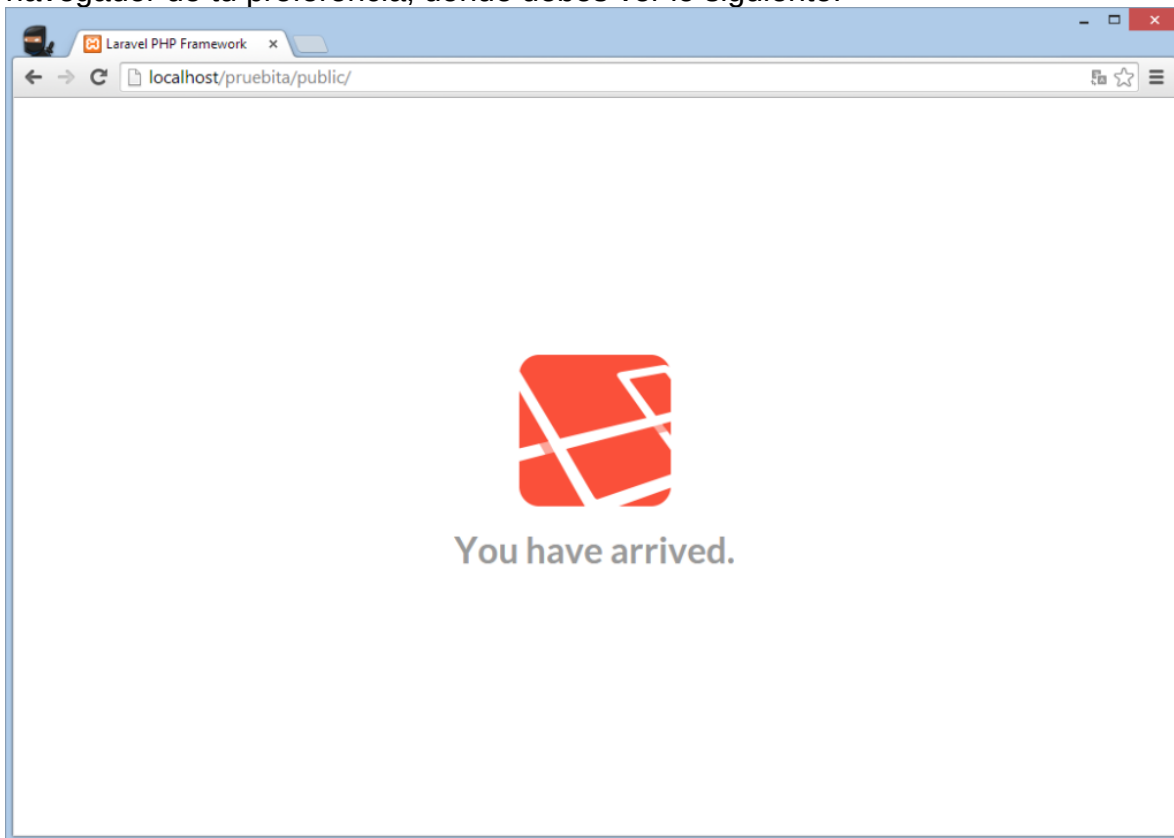
Composer empezará a descargar las librerías necesarias para nuestro proyecto, esto requiere un poco de tiempo.

```
C:\WINDOWS\system32\cmd.exe - composer create-project laravel...  
  
- Installing nikic/php-parser (v0.9.5)  
  Loading from cache  
  
- Installing jeremeamia/superclosure (1.0.1)  
  Loading from cache  
  
- Installing filp/whoops (1.1.3)  
  Loading from cache  
  
- Installing ircmaxell/password-compat (1.0.3)  
  Loading from cache  
  
- Installing d11wtq/boris (v1.0.8)  
  Loading from cache  
  
- Installing symfony/filesystem (v2.5.6)  
  Loading from cache  
  
- Installing classpreloader/classpreloader (1.0.2)  
  Loading from cache  
  
- Installing laravel/framework (v4.2.11)  
  Loading from cache
```

Si no ocurrió algún problema de conexión a Internet veremos que nuestro proyecto “pruebita” se creó correctamente.

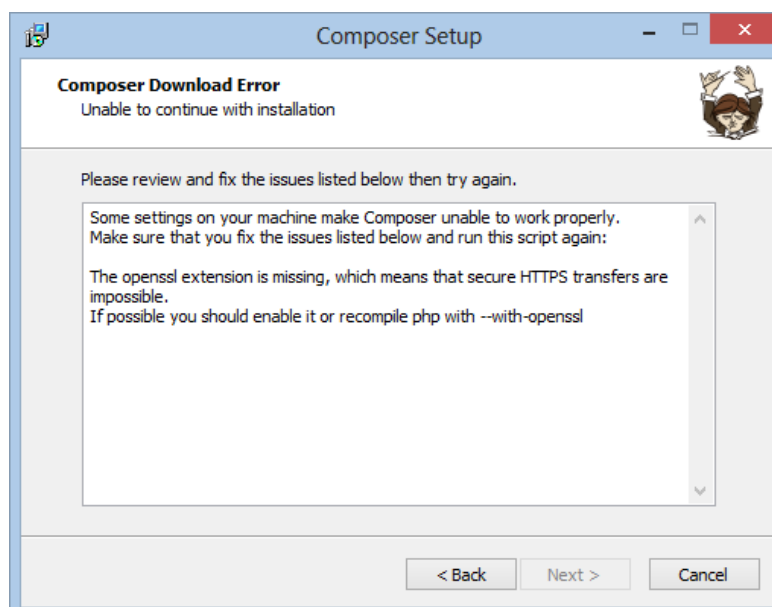
```
C:\WINDOWS\system32\cmd.exe  
  
monolog/monolog suggests installing ruflin/elastica (Allow sending log messages  
to an Elastic Search server)  
monolog/monolog suggests installing videlalvaro/php-amqplib (Allow sending log m  
essages to an AMQP server using php-amqplib)  
monolog/monolog suggests installing ext-amqp (Allow sending log messages to an A  
MQP server (1.0+ required))  
monolog/monolog suggests installing ext-mongo (Allow sending log messages to a M  
ongoDB server)  
monolog/monolog suggests installing aws/aws-sdk-php (Allow sending log messages  
to AWS services like DynamoDB)  
monolog/monolog suggests installing rollbar/rollbar (Allow sending log messages  
to Rollbar)  
d11wtq/boris suggests installing ext-readline (*)  
d11wtq/boris suggests installing ext-pcntl (*)  
d11wtq/boris suggests installing ext-posix (*)  
laravel/framework suggests installing doctrine/dbal (Allow renaming columns and  
dropping SQLite columns.)  
Writing lock file  
Generating autoload files  
Generating optimized class loader  
Compiling common classes  
Compiling views  
Application key [U5SdzVmkuBmSaIuTXtdUMzUywazKATUm] set successfully.  
  
C:\xampp\htdocs>
```

Finalmente, para verificar que la creación de nuestro proyecto “prueba” se realizó de manera correcta, accede a http://localhost/nombre_del_proyecto/public en el navegador de tu preferencia, donde debes ver lo siguiente:



Posibles errores que pueden suceder

Si utilizas **WAMPP** es posible que al ejecutar el instalador de Composer ocurra lo siguiente:

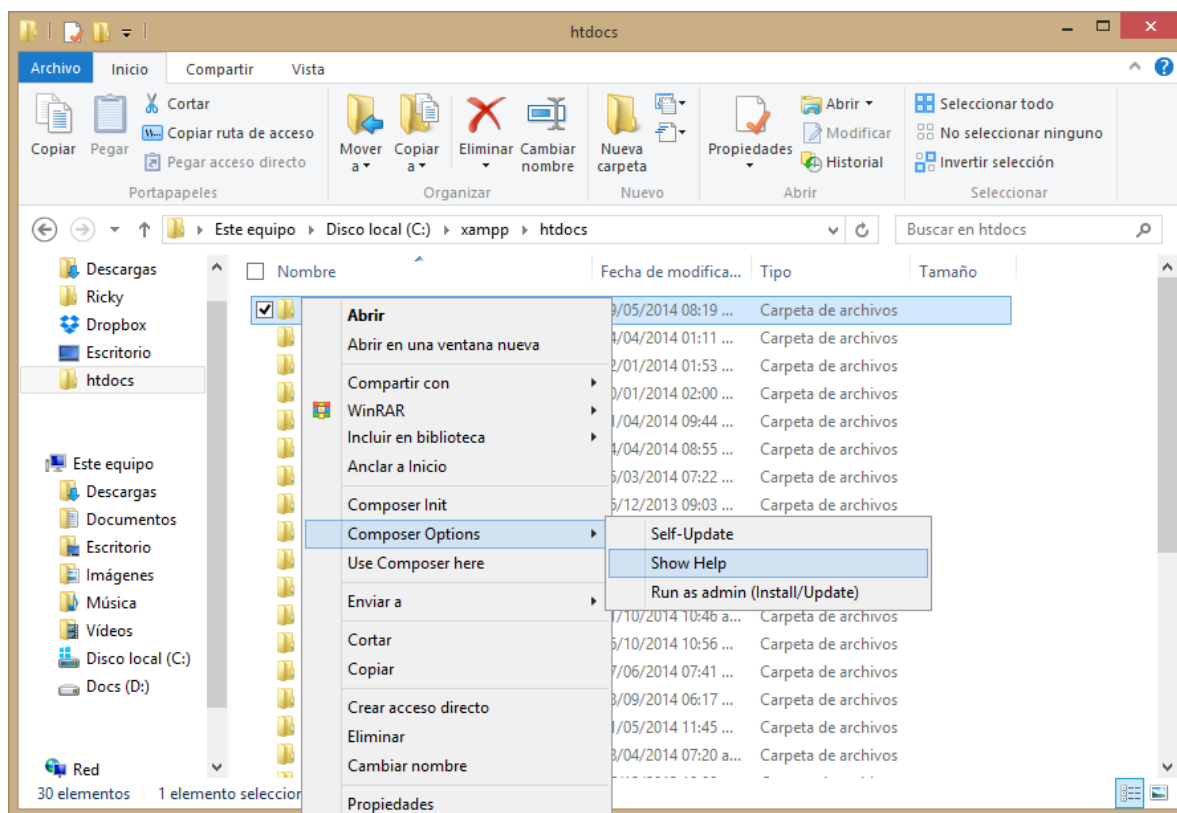


Este error lo solucionas así:

1. Ve a C:\wamp\bin\php\php5.5.12 y edita el archivo php.ini, ahora descomenta la línea extension=php_openssl.dll (quita el ; que aparece al inicio)
2. Dirígete a C:\wamp\bin\apache\Apache2.4.9\bin y edita el archivo php.ini y de nuevo descomenta la línea extension=php_openssl.dll
3. Reinicia tu servidor y ejecuta de nuevo el instalador de Composer.

Por lo general en XAMPP este error no llega a ocurrir, ya que la línea extension=php_openssl.dll viene descomentada por default.

Nota: Si decidiste seleccionar la opción “Install Shell Menus” tu Explorador de Windows se verá como la siguiente imagen:



Fuentes

<https://www.neosoft.es/blog/que-es-una-aplicacion-web/>

https://www.ecured.cu/Aplicaci%C3%B3n_web

<https://redespomactividad.weebly.com/modelo-cliente-servidor.html>

<https://styde.net/instalacion-de-composer-y-laravel-en-windows/>

<https://www.redeszone.net/windows/como-instalar-wamp-un-servidor-http-php-y-mysql-todo-en-uno/>

<http://oftgu.eco.catedras.unc.edu.ar/unidad-3/sistemas-de-gestion-de-base-de-datos/modelo-relacional-conceptos-basicos-y-fundamentos/>

<http://oftgu.eco.catedras.unc.edu.ar/unidad-3/sistemas-de-gestion-de-base-de-datos/etapas-basicas-del-diseno/>

<http://oftgu.eco.catedras.unc.edu.ar/unidad-3/sistemas-de-gestion-de-base-de-datos/caso/>

<https://www.php.net/manual/es/intro-what-is.php>